



Lecture 3 : Transformation par template avec Acceleo

Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

Générer du code (Java, SQL, HTML...) à partir d'un modèle

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi générer du texte ?** : *Pourquoi écrire un programme qui écrit du code ?*
- **Premier template** : *À quoi ressemble un template Acceleo et comment l'exécuter ?*
- **Le langage pas à pas** : *Comment parcourir le modèle : boucles, conditions, queries ?*
- **Cas complet : la bibliothèque** : *Comment générer les classes Java et le script SQL de la bibliothèque ?*
- **Bonnes pratiques et exercices** : *Quelles erreurs éviter ? Savez-vous écrire un template ?*

Objectifs de la séance

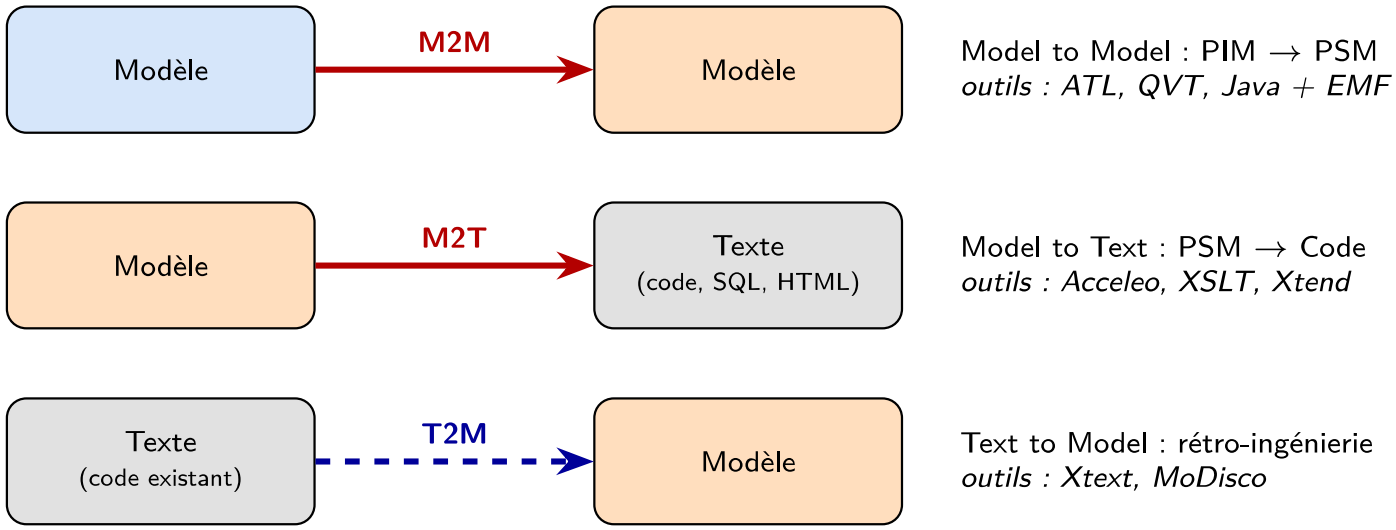
À la fin de cette séance, vous saurez :

- Expliquer le principe d'une transformation **modèle** → **texte** (M2T) **par template**.
- Lire et écrire un **module Acceleo** : `[module]`, `[template]`, `[file]`, `[for]`, `[if]`, `[query]`.
- **Naviguer** dans un modèle avec des expressions **OCL** (`m.classes`, `c.attributs->size()`...).
- Générer, à partir **d'un seul modèle**, les classes **Java** et le script **SQL** de la bibliothèque.
- Créer, exécuter et **régénérer** un projet Acceleo sous Eclipse sans perdre le code écrit à la main.

> **!** Lecture 1 (modèle, méta-modèle, Ecore) et Lecture 2 (MDA, PIM/PSM, approches de transformation).

Rappel de la Lecture 2

→ M2M, M2T, T2M



- Nous avons vu **trois approches** de transformation : par **template**, par **programmation** (Java + API EMF), par **modélisation** (ATL).
- Aujourd'hui : l'approche **par template**, pour les transformations **M2T**, avec l'outil **Acceleo**.

Pourquoi générer du texte ?

Pourquoi écrire un programme qui écrit du code ?

Pourquoi générer du texte ?

→ Le constat : beaucoup de code est mécanique

```java
public class Livre {
    private String isbn;
    private String titre;
    public String getIsbn() {
        return isbn;
    }
    public String getTitre() {
        return titre;
    }
}
```

```java
public class Adherent {
    private String nom;
    private String email;
    public String getNom() {
        return nom;
    }
    public String getEmail() {
        return email;
    }
}
```

- Les deux classes ont **exactement la même structure** : seuls les **noms** changent.
- Ces noms sont **déjà dans le modèle** (PIM) : pourquoi les recopier à la main ?

Pourquoi générer du texte ?

→ Ce que l'on gagne

Productivité

Une règle écrite **une fois** produit le code de **toutes** les classes : 3 classes ou 300, même effort.

Qualité et cohérence

Pas de faute de frappe, même style partout. Corriger le template = corriger **tous** les fichiers.

Plusieurs cibles

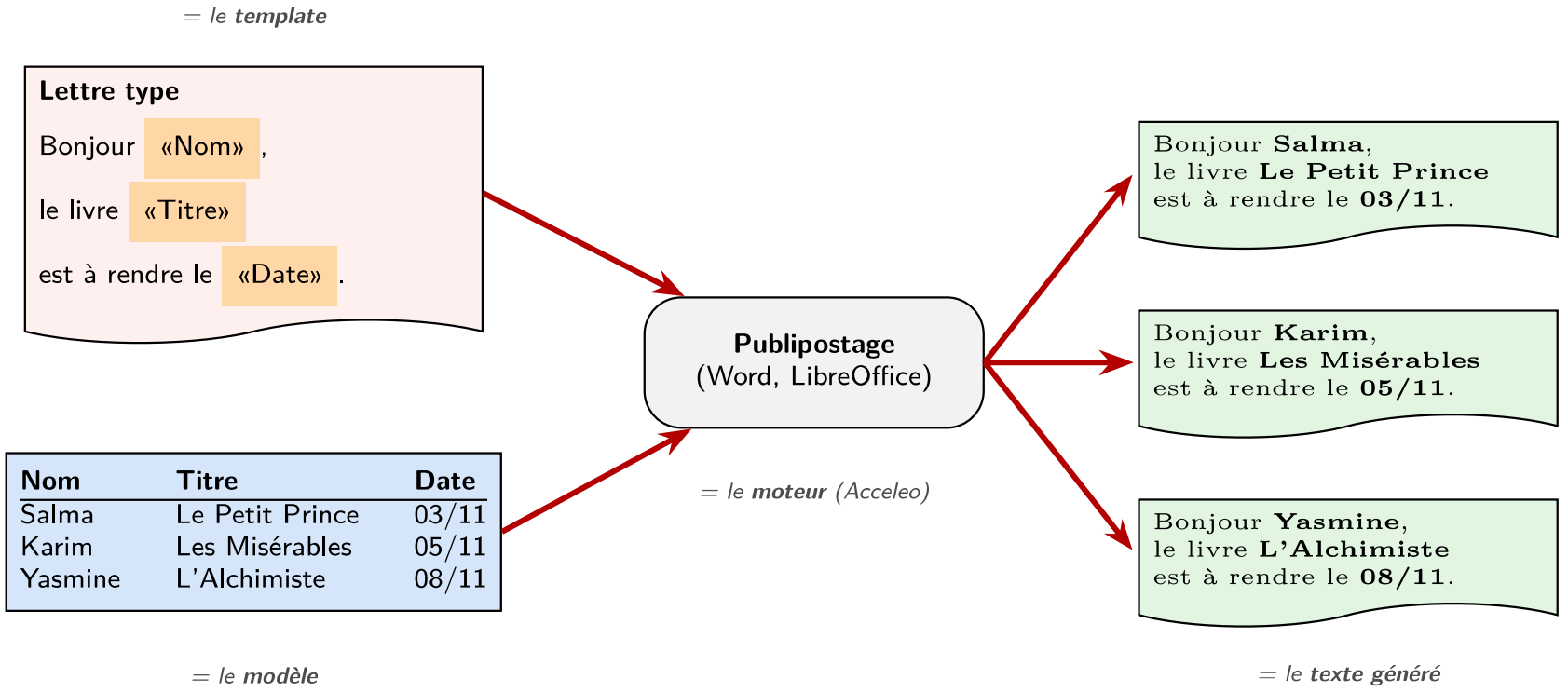
Le **même modèle** donne du Java, du SQL, du HTML, du Python. . . : un template par cible.

Suivre les évolutions

Le métier change ? On modifie **le modèle** et on **régénère**. Le code reste synchronisé avec le modèle.

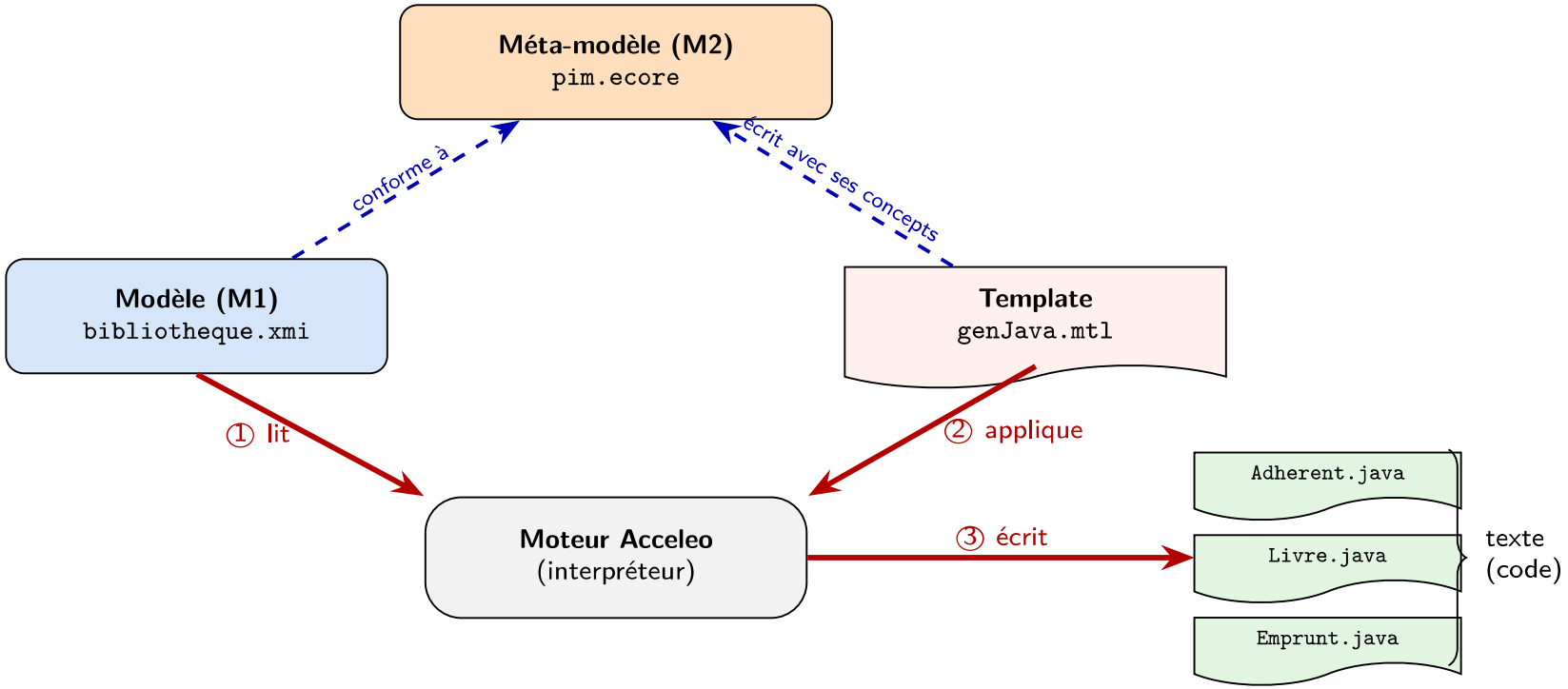
Une analogie : le publipostage

→ Vous utilisez déjà des templates !



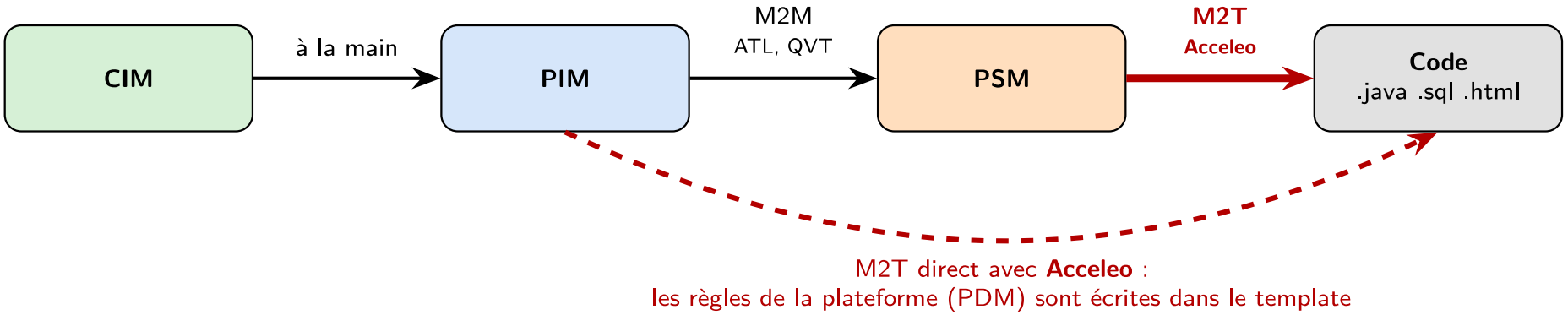
! Un **template** est un texte à trous. Le moteur remplit les trous avec les données du **modèle**, autant de fois que nécessaire.

Le principe de la transformation par template



- Le **template** (fichier `.mtl`) est écrit **une fois**, avec les concepts du **méta-modèle** (Classe, Attribut...).
- Il s'applique à **n'importe quel modèle** conforme à ce méta-modèle (bibliothèque, banque, hôpital...).

Où se situe Acceleo dans MDA ?



- Acceleo réalise la **dernière étape** de la chaîne MDA : **modèle** $\rightarrow$ **code** (transformation **M2T**, exogène, verticale).
- En pratique, on part souvent **directement du PIM** : la connaissance de la plateforme (types Java, annotations. . .) est alors placée dans le template, par exemple dans des **queries** (= le **PDM**).

Acceleo en bref

- **Acceleo** : générateur de code **open source** de la fondation **Eclipse**, créé par la société **Obeo** (France).
- Il implémente la norme **MOFM2T** (MOF Model to Text Transformation Language) de l'**OMG** (2008), l'organisme qui définit aussi UML et MDA.
- Il lit tout modèle **EMF** (fichiers `.ecore` / `.xmi`) et produit **n'importe quel texte** : Java, SQL, Python, HTML, XML, documentation...
- Les expressions entre crochets utilisent **OCL** (le langage de contraintes vu en IDM).

	Acceleo 3.x (ce cours, les TP)	**Acceleo 4.x (version actuelle)**
Langage d'expressions	OCL	AQL (proche d'OCL)
Fichier	`.mtl`	`.mtl`
En-tête de module	`[module m('uri')]`	`[module m('uri')/]`
Mode d'écriture d'un fichier	booléen : `false` = écraser	`overwrite`, `append`, `create`

Pourquoi générer du texte ?

→ Question flash

Question flash

? XSLT (Lecture 2) et Acceleo sont tous deux des langages **par template**. Quelle est la différence principale entre les deux ?

Question flash

? XSLT (Lecture 2) et Acceleo sont tous deux des langages **par template**. Quelle est la différence principale entre les deux ?

Réponse

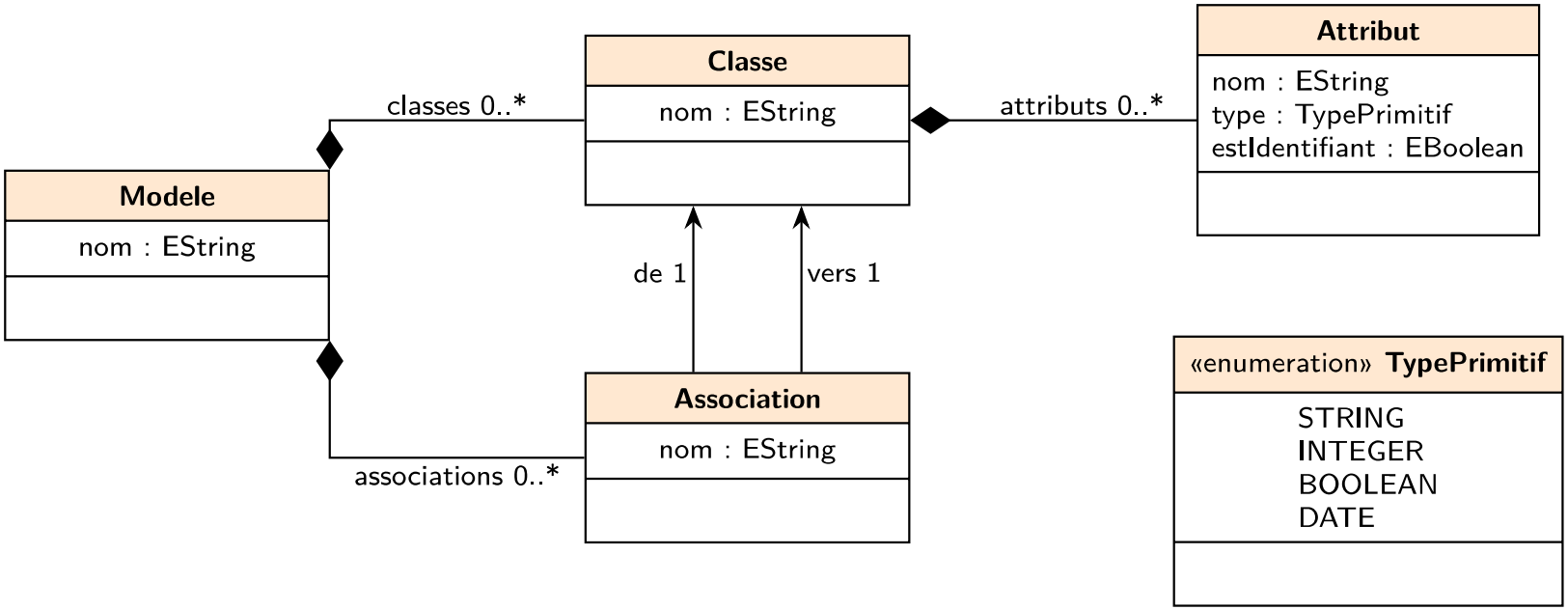
XSLT transforme des documents **XML** (les trous sont des chemins **XPath** : `<xsl:value-of select=...>`). **Acceleo** transforme des **modèles EMF** conformes à un **méta-modèle** (les trous sont des expressions **OCL** : `[c.nom/]`). Le principe « texte fixe + trous remplis par le moteur » est le même.

Premier template

À quoi ressemble un template Acceleo et comment l'exécuter ?

Le fil rouge : le méta-modèle PIM

→ Les concepts avec lesquels on écrit le template



- Fichier `pim.ecore`, URI (nsURI) : `http://ensias.ma/mda/pim`. C'est le méta-modèle de la Lecture 2.
- Un **Modele** contient des **Classes** (qui contiennent des **Attributs**) et des **Associations** (`de` → `vers`).

Le fil rouge : le modèle de la bibliothèque

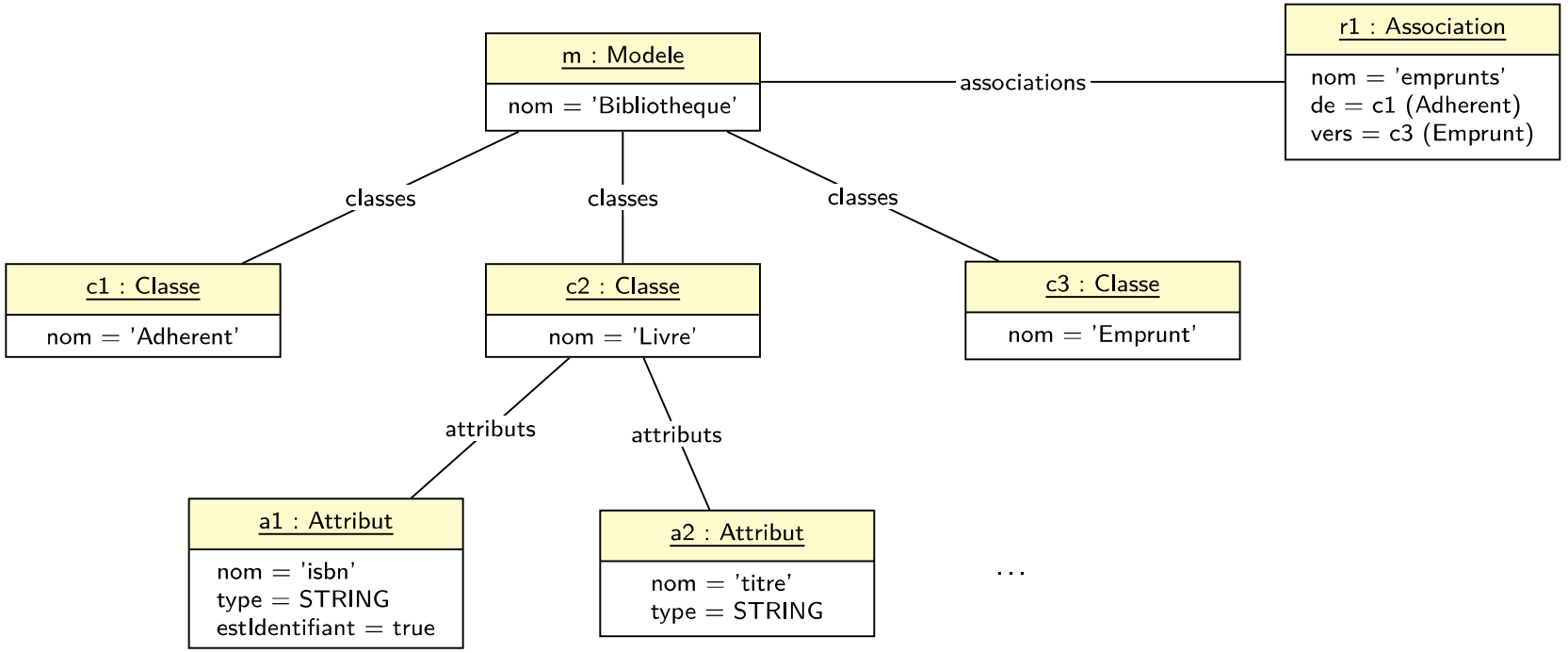
→ Le fichier bibliotheque.xmi, conforme à pim.ecore

```
<pim:Modele nom="Bibliotheque" ...>
  <classes nom="Adherent">
    <attributs nom="nom"/>
    <attributs nom="email"/>
  </classes>
  <classes nom="Livre">
    <attributs nom="isbn" estIdentifiant="true"/>
    <attributs nom="titre"/>
    <attributs nom="auteur"/>
  </classes>
  <classes nom="Emprunt">
    <attributs nom="dateDebut" type="DATE"/>
    ...
  </classes>
  <associations nom="emprunts" de="//@classes.0"
                vers="//@classes.2"/>
  <associations nom="emprunts" de="//@classes.1"
                vers="//@classes.2"/>
</pim:Modele>
```

- 3 **classes** : Adherent, Livre, Emprunt.
- Chaque classe contient ses **attributs** ; `isbn` est l'**identifiant** de Livre.
- Un attribut sans `type` a la valeur par défaut **STRING** (1er littéral de l'énumération).
- 2 **associations** : Adherent → Emprunt et Livre → Emprunt. `//@classes.2` désigne la 3e classe (Emprunt).
- C'est **ce fichier** que lira le moteur Acceleo.

Le fil rouge : le modèle de la bibliothèque

→ Le même modèle vu comme des objets (niveau M1)



! on **navigue** dans ce graphe d'objets : de `m` vers ses `classes`, puis vers leurs `attributs`, etc.

Anatomie d'un module Acceleo

→ Le premier template : une classe Java vide par classe du modèle

```
[comment encoding = UTF-8 /]
[module genJava('http://ensias.ma/mda/pim')]
```

1 **Module** : son nom et l'URI du **méta-modèle** utilisé

```
[template public genModele(m : Modele)]
[comment @main /]
```

2 **Template** : nom + paramètre typé par une méta-classe ; `@main` = point d'entrée

```
[for (c : Classe | m.classes)]
```

3 **Boucle** sur les classes du modèle

```
[file (c.nom + '.java', false, 'UTF-8')]
```

4 **Fichier** à créer (nom calculé)

```
public class [c.nom/] {
}
```

5 **Texte statique** recopié tel quel + **expression** `[ ... /]` remplacée par sa valeur

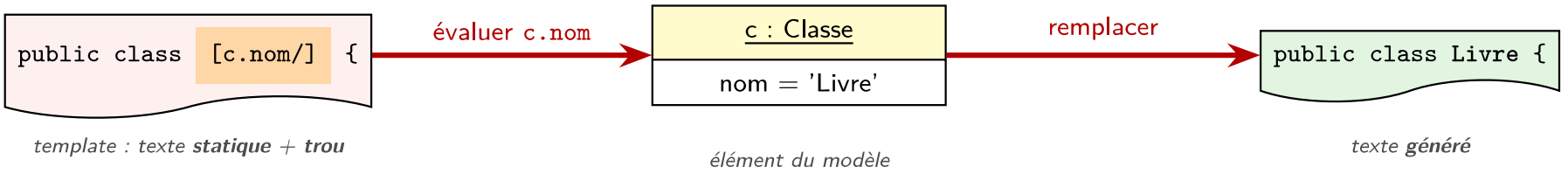
```
[/file]
[/for]
[/template]
```

6 **Fermeture** des blocs, dans l'ordre inverse

Code couleur : **violet** = déclarations, **bleu** = balises de bloc, **gris** = texte statique, **orange** = expressions.

Texte statique et expressions

→ La règle d'or d'Acceleo



- Tout ce qui est **hors des crochets** est **recopié tel quel** dans le fichier généré.
- Une **expression** `[ expression /]` est **évaluée** sur le modèle puis **remplacée par sa valeur** (attention au / final !).
- Une **balise de bloc** `[for ...]`, `[if ...]`, `[file ...]` contrôle **ce qui est écrit**, **combien de fois** et **où**. Elle se ferme par `[/for]`, `[/if]`, `[/file]`.

Résultat du premier template

→ Trois fichiers, un par classe du modèle

```
[comment encoding = UTF-8 /]
[module genJava('http://ensias.ma/mda/pim')]

[template public genModele(m : Modele)]
[comment @main /]
[for (c : Classe | m.classes)]
[file (c.nom + '.java', false, 'UTF-8')]
public class [c.nom/] {
}
[/file]
[/for]
[/template]
```

```
// Adherent.java
public class Adherent {
}
// Livre.java
public class Livre {
}
// Emprunt.java
public class Emprunt {
}
```

Le moteur a parcouru `m.classes` : 3 tours de boucle, donc **3 fichiers**.

Les balises à connaître

→ Le « vocabulaire » minimal d'Acceleo 3

Balise	Rôle	Exemple
`[module ...]`	Déclare le module et l'URI du méta-modèle	`[module genJava('http://ensias.ma/mda/pim')]`
`[template ...]`	Un bloc de génération, avec un paramètre typé	`[template public genClasse(c : Classe)]`
`[comment @main /]`	Marque le template **point d'entrée**	première ligne du template principal
`[file (...)]`	Crée un fichier ; le texte du bloc y est écrit	`[file (c.nom + '.java', false, 'UTF-8')]`
`[expr /]`	Écrit la valeur d'une expression	`[c.nom/]`, `[c.attributs->size()/]`
`[for (x : T | coll)]`	Répète le bloc pour chaque élément	`[for (a : Attribut | c.attributs)]`
`[if (cond)]`	Écrit le bloc si la condition est vraie	`[if (a.estIdentifiant)]`
`[query .../]`	Fonction réutilisable qui renvoie une valeur	`[query public javaType(t : TypePrimitif) : String = ... /]`
`[comment ... /]`	Commentaire, rien n'est généré	`[comment encoding = UTF-8 /]`

Question flash

?

On ajoute au modèle une classe **Auteur** (sans attribut). Que produit le premier template ? Que faut-il modifier dans le template ?

Question flash

? On ajoute au modèle une classe **Auteur** (sans attribut). Que produit le premier template ? Que faut-il modifier dans le template ?

Réponse

Un **quatrième fichier** `Auteur.java` contenant `public class Auteur { }`. **Rien** à modifier dans le template : il est écrit pour **toutes** les classes du méta-modèle, pas pour une classe précise. On relance simplement la génération.

Le langage pas à pas

Comment parcourir le modèle : boucles, conditions, queries ?

Naviguer dans le modèle avec OCL

→ Une expression part d'une variable et suit les références du méta-modèle

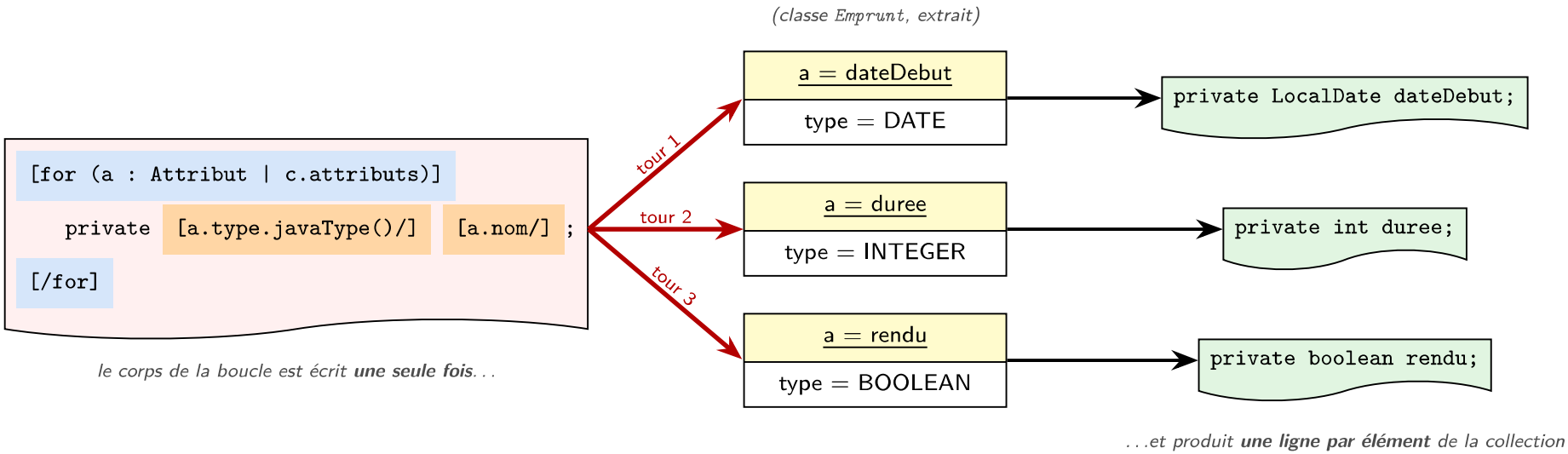
Expression	Signification	Valeur sur la bibliothèque
`m.nom`	attribut `nom` du modèle	'Bibliotheque'
`m.classes`	toutes les classes (collection)	Adherent, Livre, Emprunt
`m.classes->size()`	nombre d'éléments (-> = opération de collection)	3
`c.attributs->first().nom`	nom du 1er attribut de c (c = Livre)	'isbn'
`c.attributs->select(a | a.type = TypePrimitif::DATE)`	filtre : les attributs de type DATE (c = Emprunt)	dateDebut, dateRetour
`c.attributs->exists(a | a.estIdentifiant)`	existe-t-il un identifiant ? (c = Adherent)	false
`c.attributs->collect(a | a.nom)`	transforme : la liste des noms (c = Livre)	isbn, titre, auteur
`asso.de.nom`	suit la référence de d'une association	'Adherent'
`c.eContainer()`	l'objet qui contient c	le Modele



Le **point** . accède à une propriété d'un objet ; la **flèche** -> applique une opération à une **collection**.

La boucle [for]

→ Répéter un morceau de texte pour chaque élément



- Syntaxe : `[for (variable : Type | collection)] ... [/for]`. La variable change à **chaque tour**.
- Les boucles peuvent être **imbriquées** : classes, puis attributs de chaque classe.

La boucle [for] avec separator

→ Écrire une liste séparée par des virgules

Template

```
public [c.nom/]([for (a : Attribut | c.attributs)
    separator(', ')][a.type.javaType()/] [a.nom/][/for]) {
```

Résultat (c = Livre)

```
public Livre(String isbn, String titre, String auteur) {
```

- `separator(', ')` est écrit **entre** deux tours, jamais après le dernier : pas de virgule en trop.
- Indispensable pour les **paramètres**, les **colonnes SQL**, les **listes** JSON…

Les conditions [if]

→ Écrire un texte seulement dans certains cas

Forme complète

```
[if (a.estIdentifiant)]
    // identifiant métier
[elseif (a.type = TypePrimitif::DATE)]
    // une date
[else]
    // attribut simple
[/if]
    private [a.type.javaType()/] [a.nom/];
```

Forme en ligne

```
    private [a.type.javaType()/] [a.nom/];[if (a.
      estIdentifiant)]   // identifiant[/if]
```

Résultat pour `isbn` :

```
    private String isbn;   // identifiant
```

et pour `titre` :

```
    private String titre;
```

- Conditions possibles : `=`, `<>`, `<`, `>`, `and`, `or`, `not`, `->isEmpty()`, `->exists(...)`, `oclIsTypeOf(...)`...

Les queries

→ Des fonctions réutilisables... qui jouent le rôle du PDM

```
[query public javaType(t : TypePrimitif) : String =
    if t = TypePrimitif::STRING then 'String'
    else if t = TypePrimitif::INTEGER then 'int'
    else if t = TypePrimitif::BOOLEAN then 'boolean'
    else 'LocalDate'
    endif endif endif
/]
```

Appel : `[a.type.javaType()/]` (le 1er paramètre se place **avant** le point).

TypePrimitif (PIM)	Java	SQL	Python
STRING	String	VARCHAR(255)	str
INTEGER	int	INTEGER	int
BOOLEAN	boolean	BOOLEAN	bool
DATE	LocalDate	DATE	date



Ce tableau de correspondance des types **est** une partie du **PDM** (description de la plateforme).
Changer de plateforme = changer de query.

Opérations utiles sur les chaînes

Opération	Exemple	Résultat
`toUpperFirst()`	`'titre'.toUpperFirst()`	'Titre' (pour getTitre)
`toLowerFirst()`	`'Adherent'.toLowerFirst()`	'adherent'
`toUpper()` / `toLower()`	`'Livre'.toUpper()`	'LIVRE' (noms de tables)
`+` ou `concat()`	`c.nom + '.java'`	'Livre.java'
`size()`	`'isbn'.size()`	4
`substitute(a, b)`	`'date_debut'.substitute('_', '')`	'datedebut'
`startsWith()` / `endsWith()`	`c.nom.endsWith('s')`	true / false

Piège

Pour écrire un **crochet** dans le texte généré (liste Python `[]`, tableau Java), il faut l'écrire comme une chaîne : `['['/]` et `[']'/]`.

Variables locales : [let]

```
[let n : Integer = c.attributs->size()]
// [c.nom/] : [n/] attribut[if (n > 1)]s[/if]
[/let]
```

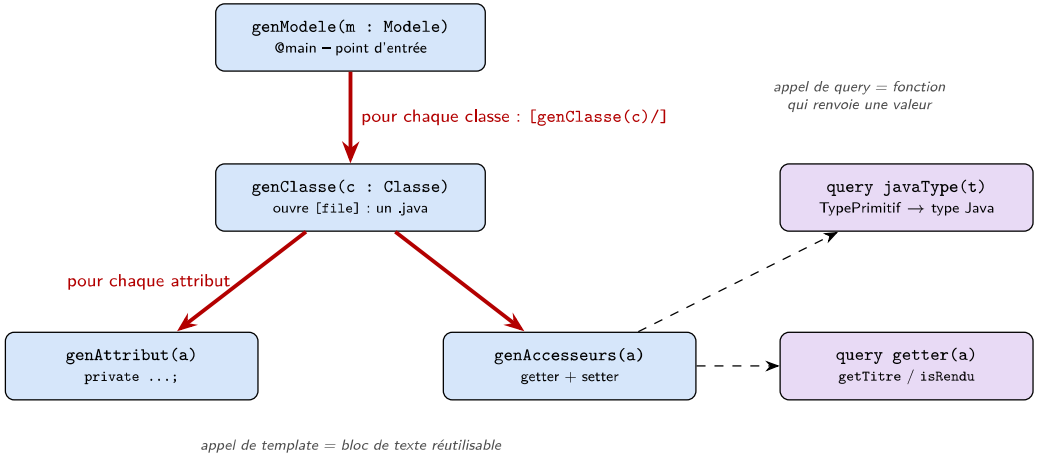
Résultat pour Livre : `// Livre : 3 attributs` ; pour une classe à un seul attribut : `// X : 1 attribut`.

- `[let]` donne un **nom** à une valeur calculée : plus lisible, et calculée une seule fois.
- La variable n'existe qu'**entre** `[let ...]` et `[/let]`.

Découper : un template peut en appeler un autre

```
[template public genModele(m : Modele)]
[comment @main /]
[for (c : Classe | m.classes)]
[genClasse(c)/]
[/for]
[/template]

[template public genClasse(c : Classe)]
[file ('bibliotheque/' + c.nom + '.java', false, 'UTF-8'
      )]
...
[/file]
[/template]
```



- Un template **appelé** s'écrit comme une expression : `[genClasse(c)/]` (ou `[c.genClasse()/]`).
- Un template = un **bloc de texte** ; une query = une **valeur** (chaîne, booléen, collection...).

Les espaces et retours à la ligne

→ Pourquoi le code généré est-il bien indenté ?

Template

```
public class [c.nom/] {
[for (a : Attribut | c.attributs)]                    ligne supprimée
   private [a.type.javaType()/] [a.nom/];
[/for]                                               ligne supprimée
}
```

Résultat

```
public class Livre {
   private String isbn;
   private String titre;
   private String auteur;
}
```

- Une ligne qui ne contient **qu'une balise de bloc** (`[for]`, `[/for]`, `[if]`, `[file]`...) ne produit **rien**, pas même un retour à la ligne.
- **Bonne pratique** : écrire les balises de bloc **en début de ligne** et n'indenter **que le texte généré**.
- Les lignes vides du template **sont** recopiées : une ligne vide en trop = une ligne vide dans le résultat.

Question flash

?

Écrire l'expression qui donne : (a) le nombre d'attributs de type DATE d'une classe c ; (b) les noms des classes, en majuscules, séparés par des points-virgules.

Question flash

?

Écrire l'expression qui donne : (a) le nombre d'attributs de type DATE d'une classe c ; (b) les noms des classes, en majuscules, séparés par des points-virgules.

Réponse

(a) `[c.attributs->select(a | a.type = TypePrimitif::DATE)->size()/]` (vaut 2 pour Emprunt). (b) `[for (c : Classe | m.classes) separator(';')][c.nom.toUpper()/][/for]` qui donne `ADHERENT;LIVRE;EMPRUNT`.

Cas complet : la bibliothèque

Comment générer les classes Java et le script SQL de la bibliothèque ?

Étape 0 : créer le projet Acceleo

→ Sous Eclipse Modeling Tools + Acceleo 3.7

- **File** > **New** > **Acceleo Project**, nom : `ma.ensias.acceleo.bibliotheque`.
- Module : `genJava` ; **Metamodel URI** : cliquer sur + et choisir `http://ensias.ma/mda/pim`.
- Type du template principal : **Modele** ; cocher **Main template** puis **Finish**.
- Copier `pim.ecore` et `bibliotheque.xmi` dans `model/` ; enregistrer le méta-modèle (clic droit sur `pim.ecore` > **Register EPackages**).

```
ma.ensias.acceleo.bibliotheque
├── src/
│   └── ma/ensias/acceleo/bibliotheque/main/
│       ├── genJava.mtl – le template (module)
│       └── GenJava.java – lanceur Java généré automatiquement
├── model/
│   ├── pim.ecore – le méta-modèle (M2)
│   └── bibliotheque.xmi – le modèle (M1)
├── src-gen/ – dossier cible : le code généré
│   └── bibliotheque/Livre.java ...
├── META-INF/MANIFEST.MF – dépendances (plug-in du méta-modèle)
└── build.properties
```

Installation

Help > **Install New Software**, site :
`https://download.eclipse.org/acceleo/updates/releases/3.7`.

Étape 1 : une classe Java par classe du modèle

→ Fichier genJava.mtl

```
[comment encoding = UTF-8 /]
[module genJava('http://ensias.ma/mda/pim')]

[template public genModele(m : Modele)]
[comment @main /]
[for (c : Classe | m.classes)]
[genClasse(c)/]
[/for]
[/template]

[template public genClasse(c : Classe)]
[file ('bibliotheque/' + c.nom + '.java', false, 'UTF-8')]
package bibliotheque;

import java.time.LocalDate;
import java.util.*;

public class [c.nom/] {
[for (a : Attribut | c.attributs)]
    private [a.type.javaType()/] [a.nom/];
[/for]
}
[/file]
[/template]
```

Résultat : Emprunt.java

```
package bibliotheque;

import java.time.LocalDate;
import java.util.*;

public class Emprunt {
    private LocalDate dateDebut;
    private LocalDate dateRetour;
    private int duree;
    private boolean rendu;
}
```

Les types viennent de la query `javaType()` (diapositive « Les queries »).

Étape 2 : constructeur et accesseurs

→ On complète genClasse et on ajoute un template et une query

```
    public [c.nom/]([for (a : Attribut | c.attributs) separator(', ')][a.type.javaType()/] [a.nom/][/for]) {
[for (a : Attribut | c.attributs)]
        this.[a.nom/] = [a.nom/];
[/for]
    }
[for (a : Attribut | c.attributs)]
[genAccesseurs(a)/]
[/for]
}

[template public genAccesseurs(a : Attribut)]

    public [a.type.javaType()/] [a.getter()/]() {
        return [a.nom/];
    }

    public void set[a.nom.toUpperFirst()/]([a.type.javaType()/] [a.nom/]) {
        this.[a.nom/] = [a.nom/];
    }
[/template]

[query public getter(a : Attribut) : String =
    (if a.type = TypePrimitif::BOOLEAN then 'is' else 'get' endif)
        .concat(a.nom.toUpperFirst())
/]
```

Étape 2 : résultat

→ Livre.java

```
public class Livre {
    private String isbn;
    private String titre;
    private String auteur;

    public Livre(String isbn, String titre, String auteur) {
        this.isbn = isbn;
        this.titre = titre;
        this.auteur = auteur;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }
    ...   // idem pour titre et auteur
}
```

- Le **constructeur** utilise `separator(',')`.
- `toUpperFirst()` construit `setIsbn`, `getTitre`...
- La query `getter()` donne `isRendu()` pour un booléen (convention Java) et `getXxx()` sinon.
- Pour 3 classes et 9 attributs : environ **120 lignes** écrites par **30 lignes** de template.

Étape 3 : identifiant et associations

→ Les queries évitent de répéter les mêmes expressions

```
public class [c.nom/] {
[if (not c.aUnIdentifiant())]
    private Long id;   // identifiant technique
[/if]
[for (a : Attribut | c.attributs)]
    private [a.type.javaType()/] [a.nom/];[if (a.estIdentifiant)
      ]   // identifiant[/if]
[/for]
[for (asso : Association | c.sortantes())]
    private List<[asso.vers.nom/]> [asso.nom/] = new ArrayList
      <>();
[/for]
[for (asso : Association | c.entrantes())]
    private [asso.de.nom/] [asso.de.nom.toLowerFirst()/];
[/for]
```

```
[query public aUnIdentifiant(c : Classe) : Boolean =
    c.attributs->exists(a | a.estIdentifiant) /]

[query public modele(c : Classe) : Modele =
    c.eContainer().oclAsType(Modele) /]

[query public sortantes(c : Classe) : Sequence(
      Association) =
    c.modele().associations->select(x | x.de = c)->
      asSequence() /]

[query public entrantes(c : Classe) : Sequence(
      Association) =
    c.modele().associations->select(x | x.vers = c)->
      asSequence() /]
```

Étape 3 : résultat

→ Les associations du modèle deviennent des attributs Java

```java
public class Emprunt {
    private Long id;   // identifiant technique
    private LocalDate dateDebut;
    private LocalDate dateRetour;
    private int duree;
    private boolean rendu;
    private Adherent adherent;
    private Livre livre;
    ...
}
```

```java
public class Livre {
    private String isbn;   // identifiant
    private String titre;
    private String auteur;
    private List<Emprunt> emprunts = new ArrayList<>();
    ...
}
```

- Emprunt n'a pas d'identifiant dans le modèle : le template ajoute un `id` **technique**.
- `emprunts` (de Livre vers Emprunt) donne une **liste** dans Livre et une **référence** `livre` dans Emprunt.

Étape 4 : un second module pour le SQL

→ Même modèle, autre plateforme : genSQL.mtl

```
[comment encoding = UTF-8 /]
[module genSQL('http://ensias.ma/mda/pim')]

[template public genScript(m : Modele)]
[comment @main /]
[file (m.nom.toLower() + '.sql', false, 'UTF-8')]
-- Script généré par Acceleo à partir du modèle [m.nom/]
[for (c : Classe | m.classes)]

CREATE TABLE [c.nom.toUpper()/] (
[if (not c.attributs->exists(a | a.estIdentifiant))]
    id BIGINT,
[/if]
[for (a : Attribut | c.attributs)]
    [a.nom/] [a.type.sqlType()/],
[/for]
[for (asso : Association | m.associations->select(x | x.vers = c))]
    [asso.de.nom.toLower()/]_id [asso.de.cleSql()/],
[/for]
    PRIMARY KEY ([c.cle()/])[for (asso : Association | m.associations->select(x | x.vers = c))],
    FOREIGN KEY ([asso.de.nom.toLower()/]_id) REFERENCES [asso.de.nom.toUpper()/]([asso.de.cle()/])[/for]
);
[/for]
[/file]
[/template]
```

Le `[for]` **en ligne** après `PRIMARY KEY` ajoute « , FOREIGN KEY . . . » **seulement** s'il y a des associations entrantes.

Étape 4 : les queries du module SQL

→ Le PDM « relationnel »

```
[query public sqlType(t : TypePrimitif) : String =
    if t = TypePrimitif::STRING then 'VARCHAR(255)'
    else if t = TypePrimitif::INTEGER then 'INTEGER'
    else if t = TypePrimitif::BOOLEAN then 'BOOLEAN'
    else 'DATE' endif endif endif /]

[query public cle(c : Classe) : String =
    if c.attributs->exists(a | a.estIdentifiant)
    then c.attributs->any(a | a.estIdentifiant).nom
    else 'id' endif /]

[query public cleSql(c : Classe) : String =
    if c.attributs->exists(a | a.estIdentifiant)
    then c.attributs->any(a | a.estIdentifiant).type.sqlType()
    else 'BIGINT' endif /]
```

- `sqlType()` : correspondance des types (comme `javaType()` pour Java).
- `cle()` / `cleSql()` : nom et type de la clé primaire (l'identifiant du modèle, sinon `id BIGINT`).
- `any(...)` renvoie **un** élément qui vérifie la condition.

Étape 4 : résultat

→ bibliotheque.sql

```sql
-- Script généré par Acceleo à partir du modèle Bibliotheque

CREATE TABLE ADHERENT (
    id BIGINT,
    nom VARCHAR(255),
    email VARCHAR(255),
    PRIMARY KEY (id)
);

CREATE TABLE LIVRE (
    isbn VARCHAR(255),
    titre VARCHAR(255),
    auteur VARCHAR(255),
    PRIMARY KEY (isbn)
);

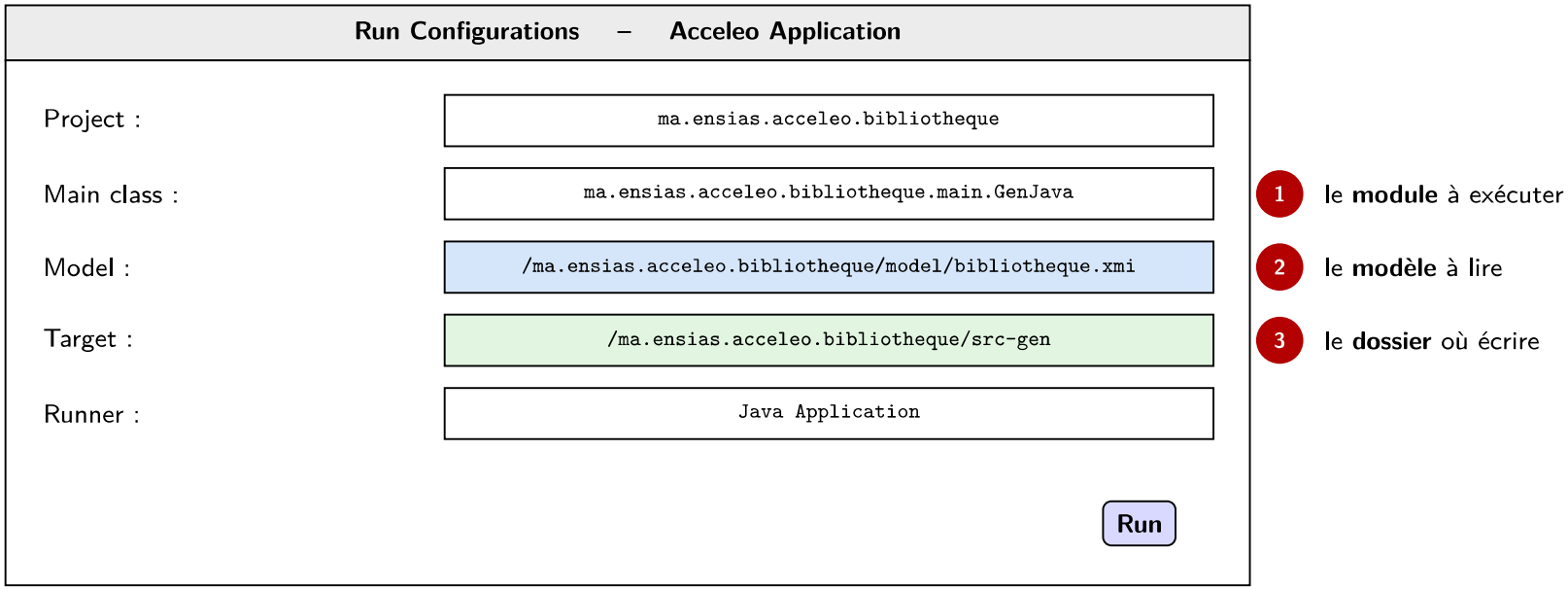
CREATE TABLE EMPRUNT (
    id BIGINT,
    dateDebut DATE,
    dateRetour DATE,
    duree INTEGER,
    rendu BOOLEAN,
    adherent_id BIGINT,
    livre_id VARCHAR(255),
    PRIMARY KEY (id),
    FOREIGN KEY (adherent_id) REFERENCES ADHERENT(id),
    FOREIGN KEY (livre_id) REFERENCES LIVRE(isbn)
);
```

- Classe → **table** ; attribut → **colonne** ; type → type SQL (query `sqlType`).
- Identifiant → **PRIMARY KEY** ; sinon colonne `id` technique.
- Association `de` → `vers` → **clé étrangère** dans la table `vers`.
- Ce sont les règles R1 à R5 de la Lecture 2. . . écrites cette fois **par template**.

genJava.mtl et **genSQL.mtl** lisent le **même** `bibliotheque.xmi`.

Exécuter la génération

→ Run As > Launch Acceleo Application



- Clic droit sur `genJava.mtl` > **Run As** > **Launch Acceleo Application**, puis remplir **Model** et **Target**.
- Les fichiers apparaissent dans `src-gen/` (faire **F5** / **Refresh** si besoin). On relance après **chaque** modification du modèle ou du template.

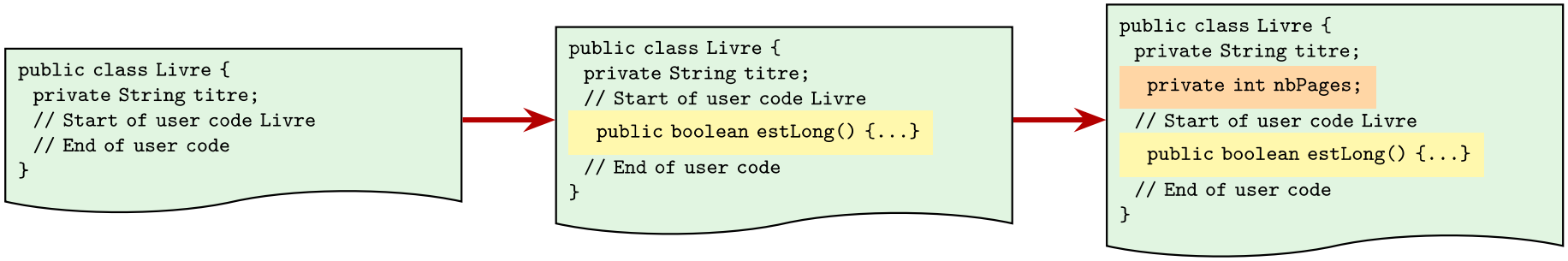
Régénérer sans perdre son code : les zones protégées

Template

```
public class [c.nom/] {
    ...
    // [protected ('methodes ' + c.nom)]
    // Écrivez ici votre code : il sera conservé
    // [/protected]
}
```

Code généré

```
public class Livre {
    ...
    // Start of user code methodes Livre
    // Écrivez ici votre code : il sera conservé
    // End of user code
}
```



① *1re génération*

② *le développeur écrit **dans** la zone protégée*

③ *on ajoute `nbPages` au modèle et on **régénère** : la zone est **conservée***

Le texte écrit **entre** `Start of user code` et `End of user code` est **conservé** à chaque régénération (l'identifiant doit être **unique** dans le fichier).

Changer de plateforme = changer de template

→ Le même PIM projeté vers Java / JPA

```
@Entity
public class [c.nom/] {
[if (not c.aUnIdentifiant())]
    @Id @GeneratedValue
    private Long id;
[/if]
[for (a : Attribut | c.attributs)]
[if (a.estIdentifiant)]
    @Id
[/if]
    private [a.type.javaType()/] [a.nom/];
[/for]
[for (asso : Association | c.sortantes())]
    @OneToMany(mappedBy = "[c.nom.toLowerFirst()/]")
    private List<[asso.vers.nom/]> [asso.nom/] = new ArrayList<>();
[/for]
[for (asso : Association | c.entrantes())]
    @ManyToOne(optional = false)
    private [asso.de.nom/] [asso.de.nom.toLowerFirst()/];
[/for]
```

On garde le **même modèle** et on change **quelques lignes** du template (annotations `@Entity`, `@Id`, `@OneToMany`...) : c'est l'idée de MDA (PIM → plusieurs PSM / codes).

Variante : générer depuis un fichier .ecore

→ Le méta-modèle est alors Ecore lui-même (tutoriel Library)

Notre PIM	**Ecore (`http://www.eclipse.org/emf/2002/Ecore`)**
Modele	EPackage
`m.classes`	`p.eClassifiers->filter(EClass)`
Classe / `c.nom`	EClass / `cls.name`
`c.attributs`	`cls.eAttributes`
`a.type`	`a.eAttributeType.name` ('EString', 'EInt'...)
Association	EReference (`cls.eReferences`)

`filter(EClass)` garde les EClass (et pas les EEnum, EDataType). Équivalent : `select(c | c.oclIsKindOf(EClass))`.

Attention

Le paramètre du template principal est alors un **EPackage** (la racine d'un fichier `.ecore`).

Variante : le template Python

→ library2python.mtl (version corrigée du tutoriel)

```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')]

[template public genPython(p : EPackage)]
[comment @main /]
[file (p.name + '.py', false, 'UTF-8')]
# Code Python généré à partir de [p.name/].ecore
[for (cls : EClass | p.eClassifiers->filter(EClass))]

class [cls.name/]:
    def __init__(self[for (a : EAttribute | cls.eAttributes)], [a.name/]=None[/for]):
[for (a : EAttribute | cls.eAttributes)]
        self.[a.name/] = [a.name/]
[/for]
[for (r : EReference | cls.eReferences)]
        self.[r.name/] = ['['/][']'/]
[/for]
[/for]
[/file]
[/template]
```

Variante : résultat Python

→ library.ecore (Library, Book, Member, Loan) → library.py

```
class Book:
    def __init__(self, id=None, title=None, isbn=None, author=None, available=None):
        self.id = id
        self.title = title
        self.isbn = isbn
        self.author = author
        self.available = available

class Loan:
    def __init__(self, loanDate=None, returnDate=None):
        self.loanDate = loanDate
        self.returnDate = returnDate
        self.member = []
        self.book = []
```

- `['['/]['']'/]` écrit `[]` : une liste Python vide pour chaque référence.
- Même principe, autre méta-modèle : les noms des propriétés (`name`, `eAttributes`...) sont ceux d'**Ecore**.

Bonnes pratiques et exercices

Quelles erreurs éviter ? Savez-vous écrire un template ?

Les erreurs les plus fréquentes

Symptôme	Cause probable	Solution
Aucun fichier généré	pas de `[comment @main /]`, ou mauvais type de paramètre du main	mettre `@main` dans le template dont le paramètre est la **racine** du modèle (Modele)
`[c.nom]` recopié tel quel	oubli du `/` final	écrire `[c.nom/]`
« Unresolved type Classe »	URI du méta-modèle fausse ou méta-modèle non enregistré	vérifier le `nsURI` du `.ecore` ; Register EPackages
Erreur à la fermeture	blocs mal imbriqués (`[/if]` au lieu de `[/for]`)	fermer dans l'**ordre inverse** de l'ouverture
Virgule en trop	virgule écrite dans le corps du `[for]`	utiliser `separator(', ')`
Code manuel perdu	modification du fichier généré **hors** zone protégée	`[protected]`, ou modifier le **template**
Fichiers dans un dossier inattendu	chemin relatif au **Target**	choisir le Target ; chemin dans `[file]`

Template, programmation ou modélisation ?

→ Retour sur les approches de la Lecture 2

	Template (Acceleo)	**Programmation (Java + EMF)**	**Modélisation (ATL)**
Résultat	**texte** (M2T)	texte ou modèle	**modèle** (M2M)
Ce qu'on écrit	le texte cible avec des **trous**	du code qui parcourt le modèle et construit le texte (`StringBuilder`)	des **règles** source → cible
Lisibilité	très bonne : on « voit » le résultat	moyenne : le texte est noyé dans le Java	bonne
Usage typique	code, SQL, HTML, documentation	traitements spécifiques, outils	PIM → PSM



Pour **produire du texte**, préférez un template : le template **ressemble** au code généré.

Exercice 1 : générer la documentation HTML

→ Énoncé

Écrire un module `genDoc.mtl` qui produit, pour le modèle de la bibliothèque, le fichier `doc/Bibliotheque.html` suivant :

```
<html><body>
<h1>Modèle Bibliotheque</h1>
<h2>Adherent (2 attributs)</h2>
<ul>
  <li>nom : STRING</li>
  <li>email : STRING</li>
</ul>
<h2>Livre (3 attributs)</h2>
<ul>
  <li>isbn : STRING <b>(identifiant)</b></li>
  ...
```

- Un titre `<h1>` pour le modèle, un `<h2>` par classe avec son **nombre** d'attributs, une liste `<li>` par attribut.
- Les identifiants sont signalés par `<b>(identifiant)</b>`.

Exercice 1 : correction

```
[comment encoding = UTF-8 /]
[module genDoc('http://ensias.ma/mda/pim')]

[template public genDoc(m : Modele)]
[comment @main /]
[file ('doc/' + m.nom + '.html', false, 'UTF-8')]
<html><body>
<h1>Modèle [m.nom/]</h1>
[for (c : Classe | m.classes)]
<h2>[c.nom/] ([c.attributs->size()/] attributs)</h2>
<ul>
[for (a : Attribut | c.attributs)]
  <li>[a.nom/] : [a.type/][if (a.estIdentifiant)] <b>(identifiant)</b>[/if]</li>
[/for]
</ul>
[/for]
</body></html>
[/file]
[/template]
```

`[a.type/]` écrit le nom du littéral de l'énumération (STRING, DATE...). Le `[if]` **en ligne** n'ajoute le texte que pour l'identifiant.

Exercice 2 : trouver les 6 erreurs

```
[module genJava('http://ensias.ma/mda/pim')/]
[template public genClasse(c : Classe)]
[file (c.nom + '.java', false, 'UTF-8')]
public class [c.nom] {
[for (a : Attribut | c.attribut)]
    private [a.type.javaType()/] [a.nom/]
[/if]
}
[/template]
```

Question flash

? Ce module Acceleo 3 contient **6 erreurs** (Acceleo ou Java). Lesquelles ?

Exercice 2 : trouver les 6 erreurs

```
[module genJava('http://ensias.ma/mda/pim')/]
[template public genClasse(c : Classe)]
[file (c.nom + '.java', false, 'UTF-8')]
public class [c.nom] {
[for (a : Attribut | c.attribut)]
    private [a.type.javaType()/] [a.nom/]
[/if]
}
[/template]
```

Question flash

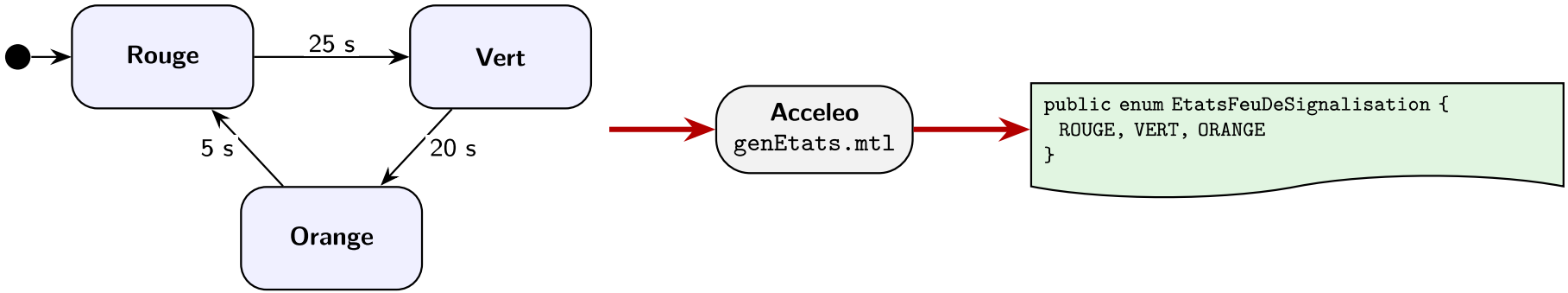
? Ce module Acceleo 3 contient **6 erreurs** (Acceleo ou Java). Lesquelles ?

Réponse

(1) `[module ...]` : pas de / en Acceleo 3. (2) `[c.nom]` → `[c.nom/]`. (3) `c.attribut` → `c.attributs`. (4) ; manquant après `[a.nom/]` (erreur dans le **Java** généré). (5) `[/if]` → `[/for]`. (6) `[/file]` manquant avant `[/template]`.

Exercice 3 : machine à états → Java

→ Énoncé



modèle `feu.xmi` (conforme à `sm.ecore`)

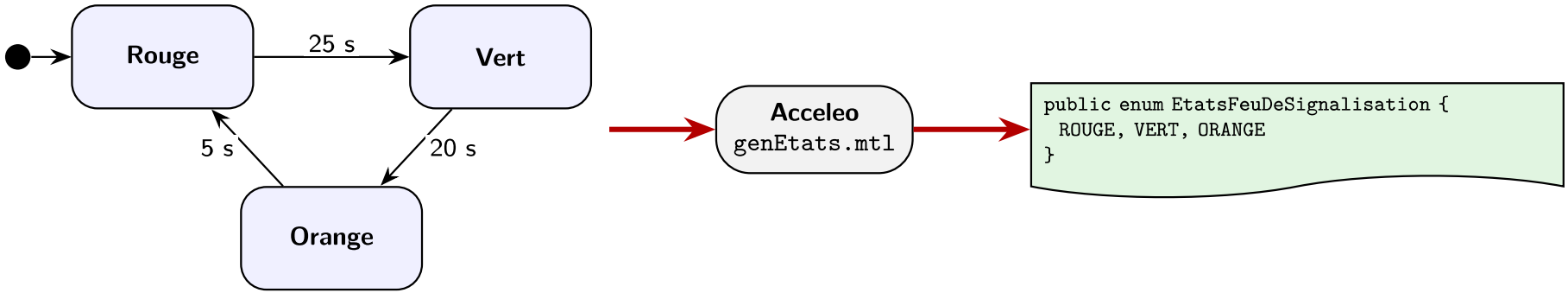
Avec le méta-modèle `sm.ecore` (StateMachine, Vertex, State, InitialState, FinalState, Transition), écrire `genEtats.mtl` qui produit l'énumération Java des **états normaux** (State) de la machine.

Question flash

? Quelle expression OCL sélectionne uniquement les états de type **State** parmi `sm.vertices` ?

Exercice 3 : machine à états → Java

→ Énoncé



modèle feu.xmi (conforme à sm.ecore)

Avec le méta-modèle `sm.ecore` (StateMachine, Vertex, State, InitialState, FinalState, Transition), écrire `genEtats.mtl` qui produit l'énumération Java des **états normaux** (State) de la machine.

Question flash

? Quelle expression OCL sélectionne uniquement les états de type **State** parmi `sm.vertices` ?

Réponse

`sm.vertices->select(v | v.oclIsTypeOf(State))` (ou `sm.vertices->filter(State)`). `oclIsTypeOf` exclut l'état initial et l'état final, qui sont aussi des Vertex.

Exercice 3 : correction

```
[comment encoding = UTF-8 /]
[module genEtats('http://ensias.ma/idm/sm')]

[template public genEnum(sm : StateMachine)]
[comment @main /]
[file ('Etats' + sm.name + '.java', false, 'UTF-8')]
public enum Etats[sm.name/] {
    [for (s : Vertex | sm.vertices->select(v | v.oclIsTypeOf(State)))
        separator(', ')][s.name.toUpper()/][/for]
}
[/file]
[/template]
```

- `toUpper()` met les constantes en majuscules (convention Java), `separator(', ')` les sépare.
- Pour aller plus loin (TP) : générer aussi une méthode `suivant()` avec un `switch` construit à partir des **transitions**.

À retenir

Template

Un texte **à trous** : texte statique + expressions `[ ... /]` évaluées sur le modèle.

Module Acceleo

`[module]` + URI du méta-modèle, `[template]` avec `@main`, `[file]` pour créer les fichiers.

Navigation OCL

`.` pour une propriété, `->` pour une collection : `select`, `exists`, `size`, `collect...`

Contrôle

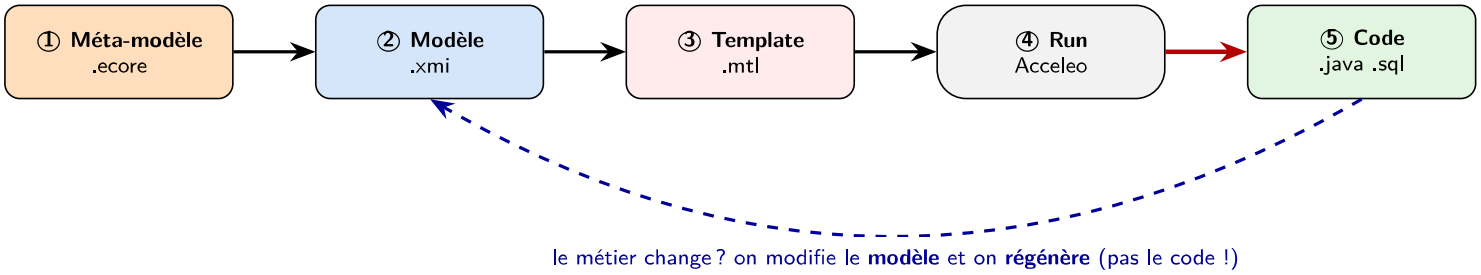
`[for]` (+ `separator`), `[if]/[elseif]/[else]`, `[let]`, appels de templates.

Queries = PDM

Les règles de la plateforme (types, noms) dans des **queries** réutilisables.

Régénérer

On modifie le **modèle**, pas le code ; le code manuel va dans les **zones protégées**.



Glossaire

Terme	Définition courte
M2T	Model to Text : transformation d'un modèle en texte (code, SQL, HTML...)
Template	texte modèle contenant des parties fixes et des « trous » calculés
Acceleo / MOFM2T	générateur de code Eclipse / norme OMG des langages modèle → texte
Module (.mtl)	fichier Acceleo : déclare le méta-modèle et contient templates et queries
@main	marque le template point d'entrée de la génération
Query	fonction OCL réutilisable qui renvoie une valeur
Zone protégée	partie du fichier généré conservée lors des régénérations
Target	dossier où sont écrits les fichiers générés

Prochaines séances

TD : lire, compléter et corriger des templates (Java, SQL, Python, machine à états). **TP** : projet Acceleo complet sur la bibliothèque (Java, SQL, toString, zones protégées, machine à états).